# Plan d'infrastructure OSINT

## Une architecture commune pour la veille OSINT

Ce plan décrit l'architecture de données du corpus TASS. L'analyste retrouve un article et vérifie son texte ; le responsable de veille contrôle les sources et les limites de la synthèse. PostgreSQL, MongoDB et un index Elasticsearch relié à Kibana partagent les mêmes identifiants.

| Disponible et exécuté | Limites explicites |
|---|---|
| 21 681 articles au raccordement : 21 676 initiaux et 5 nouveaux ; trois essais synthétiques. | Un poste local, une seule source et aucun nouveau modèle NER. |
| TLS, rôles, chiffrement des textes, sauvegarde et restauration de contrôle. | Métadonnées et corpus source non entièrement chiffrés au repos ; copie hors machine absente. |
| Interfaces, code, captures et vidéo de lecture avec panne simulée. | La vidéo démontre une exécution locale, pas une exploitation externe de production. |

## Sommaire


| Rubrique | Page |
|---|---|
| Cahier des charges | 2 |
| Schémas SQL et NoSQL | 3 et 4 |
| Architecture et Compose | 5 et 6 |
| Sécurité et confidentialité | 7 |
| Captures et supervision | 8 à 10 |
| Volumétrie et résultats | 11 |
| Sauvegarde, continuité et droits | 12 et 13 |
| Utilisation et maintenance | 14 |
| Écarts, sources et oral | 15 et 16 |
| Cartographie et raccordement | 17 et 18 |
| Correspondance ANSSI | 19 et 20 |

# 1. Cahier des charges

## Contexte et usages

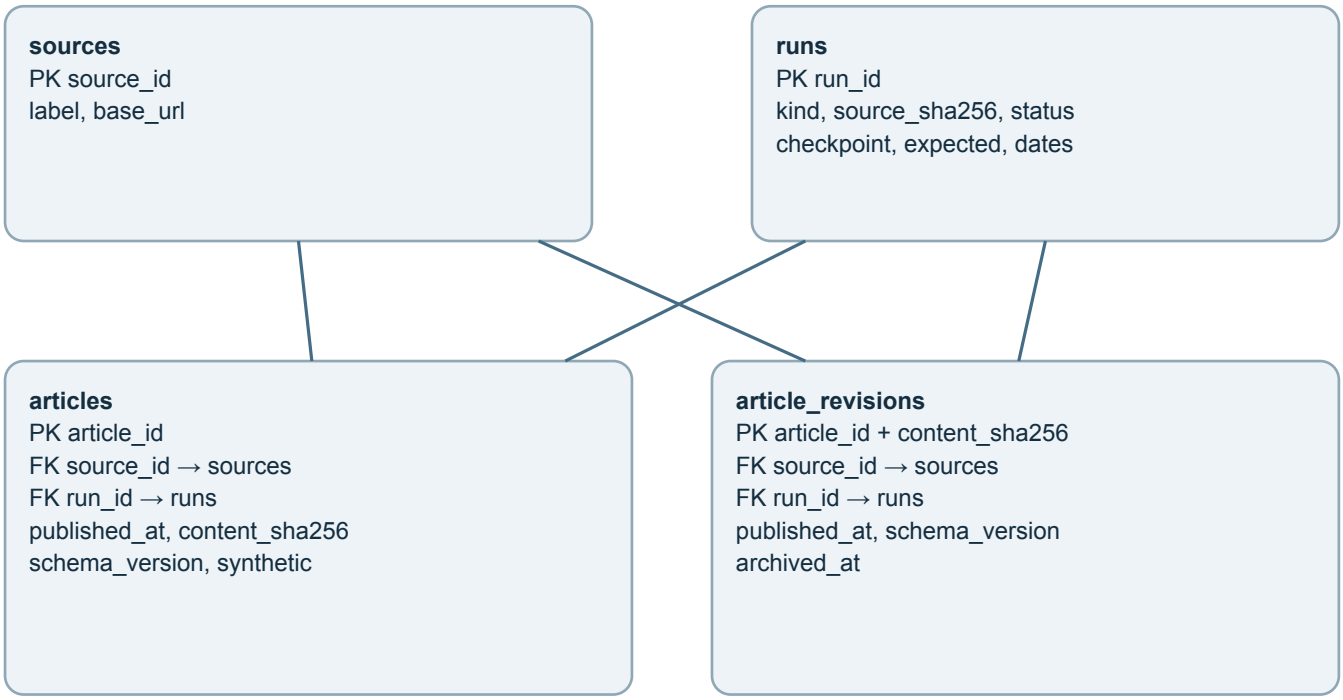
Le nettoyage conserve 21 676 des 21 742 entrées initiales. Le programme développé dans le Bloc 3 enrichit ce corpus par rubrique et période UTC ; Bloc 2 importe les lots validés. Le raccordement vérifié compte 21 681 articles. Une collecte ne déclenche ni inférence ni entraînement.

| Besoin | Réponse et contrôle |
|---|---|
| Données hétérogènes | Métadonnées relationnelles, JSON variable et texte intégral. Contrats SQL et MongoDB vérifiés. |
| Traçabilité | Identifiant partagé, empreintes du texte et du JSON, provenance, exécution, point de reprise et révisions conservées. |
| Lecture et source | API HTTPS authentifiée : article et liste par dates UTC, au plus 1 000 résultats. Index NER Bloc 4 distinct, limité aux textes déjà analysés. |
| Cadence proposée | Hypothèse de dimensionnement : un lot quotidien de 1 000 nouveaux articles, disponible en moins de 15 minutes. Ces cibles sont des choix du scénario. |
| Charge de référence | 21 681 articles au raccordement ; 21 676 pendant les essais de charge. Trois tailles synthétiques : 60 000, 600 000 et 6 000 000 enregistrements dans chaque base. |
| Disponibilité | Lecture dégradée depuis l'index, reprise d'ingestion et restauration. Pas de continuité garantie en cas de panne de l'hôte. |
| Ressources | Docker Desktop : 11 CPU visibles, environ 8,2 Go de mémoire. Limites : PostgreSQL 640 Mio, MongoDB 768 Mio, ingestion 512 Mio. |

Extension : une nouvelle source doit être inscrite dans la table sources ; ses documents peuvent porter source_id et des champs JSON complémentaires. Un changement de schéma exige une migration explicite. La conservation de champs optionnels imbriqués a été vérifiée sur une fixture distincte du corpus.

# 2. Schéma SQL PostgreSQL

L'ERD opérationnel est retenu ; aucun entrepôt en étoile n'est implémenté. Cinq tables séparent sources, exécutions, articles, effacements et révisions. sql/01_schema.sql et la migration additive sql/02_revisions.sql définissent le schéma ; docs/sql_schema.dbml décrit aussi les archives.




Les révisions archivent les métadonnées précédentes quand le contenu change. La clé composée empêche une archive dupliquée au rejeu. Les index de date servent la sélection temporelle ; une migration additive conserve les volumes.

| Table complémentaire | Rôle |
|---|---|
| erasures | article_id, request_id, requested_at, status. Registre conservé après effacement, sans dépendance obligeant à garder le texte. |

API /api/articles : dates inclusives UTC, réalisées par published_at >= début et < lendemain de fin. Réponse limitée à 1 000 métadonnées, sans pagination. Les dates de collecte et de publication restent distinctes.

Le compte osint_reader lit uniquement ; osint_writer modifie les tables applicatives sans être superutilisateur. Preuves/Revisions_collecte_TASS.json vérifie révisions, rejeu, dates et effacement.

# 2. Schéma documentaire MongoDB

Un document conserve l'identifiant SQL, la date, une empreinte et le JSON chiffré. MongoDB fait autorité pour le contenu textuel ; la publication dépend aussi de la validation SQL.

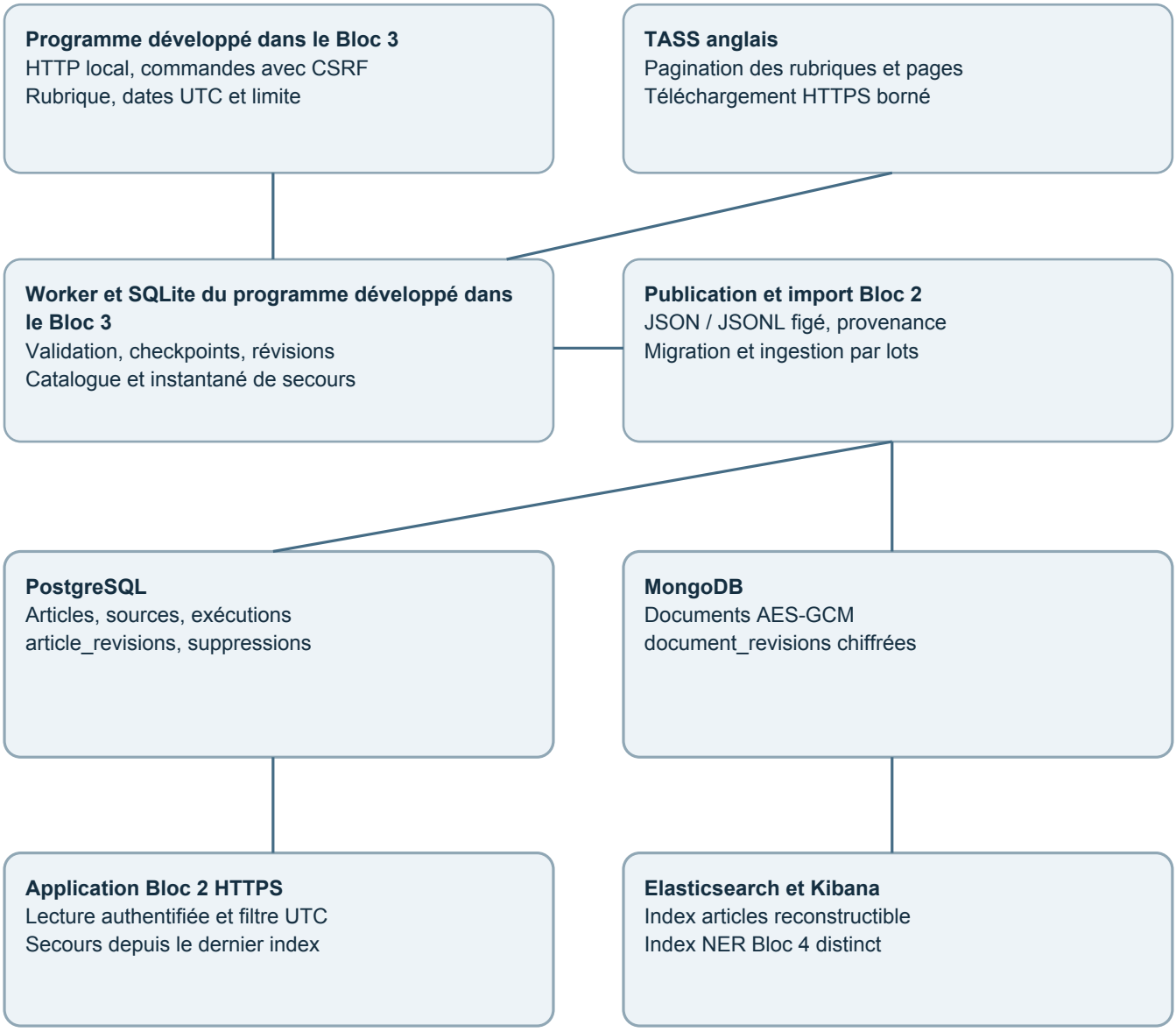
```
{
  "_id": "2035207",
  "published_at": "2025-10-26T09:20:23Z",
  "ciphertext": "<nonce et contenu AES-GCM encodés>",
  "content_sha256": "<SHA-256 du JSON en clair>",
  "schema_version": 1,
  "run_id": "<UUID de l'exécution>",
  "synthetic": false
}
```

Ce schéma est un exemple explicatif, pas une nouvelle donnée mesurée. Le JSON avant chiffrement contient title, text et url, ainsi que les champs optionnels de la source. La clé de chiffrement n'est jamais stockée avec le document.

| Contrôle | Mécanisme |
|---|---|
| Types et champs | Validateur MongoDB dans mongo/init.js ; rejet d'un document incomplet testé. |
| Index | _id unique ; published_at ; run_id. |
| Lien entre moteurs | Même identifiant et même empreinte. Contrôle du corpus initial, puis concordance des 5 nouveaux articles dans SQL, MongoDB et Elasticsearch : dates, textes, provenance et empreintes. |
| Écriture partielle | MongoDB est écrit avant le commit SQL. Un lot incomplet se rejoue sans doublon ; il n'est pas présenté comme une transaction distribuée. |
| Révisions | document_revisions archive le ciphertext précédent, article_id, empreinte et archived_at. Clé _id : article_id:empreinte ; déchiffrement avec article_id original. |
| Contrat | schema_version=1 ; révision de contenu distincte de la version du schéma. Texte et offsets Unicode conservés. |

# 3. Architecture et flux

Le navigateur pilote un service local du programme développé dans le Bloc 3 et un worker séparé. Le flux Bloc 2 passe par les conteneurs opérateurs ; aucun secret des bases ne transite dans le navigateur.




Le corpus nettoyé initial reste une autre entrée d'ingestion. PostgreSQL valide les identifiants et métadonnées ; MongoDB garde les textes. Une divergence doit être résolue avant indexation. Aucune transaction distribuée SQL/MongoDB.

Le raccordement conserve 39 511 mentions NER sur le corpus initial. Les cinq nouveaux articles ne sont pas analysés dans cette recette. Le schéma détaillé est dans docs/architecture.mmd.

# 4. Compose commenté

Le projet porte le nom memoire_osint. Les volumes nommés sont dédiés au projet. Les images sont fixées par empreinte ; les versions Elastic sont explicites et relevées dans les preuves.

```
postgres:
  volumes:
    - pg_data:/var/lib/postgresql/data
  secrets: [pg_admin, writer, reader, postgres.key, postgres.crt]
  healthcheck:
    test: [CMD-SHELL, 'pg_isready -U postgres -d osint']
  networks: [data_net]
app:
  ports: ['127.0.0.1:${API_PORT:-18443}:8443']
  depends_on:
    postgres: {condition: service_healthy}
    mongo: {condition: service_healthy}
networks:
  front_net:
  data_net:
    internal: true
```

| Choix | Pourquoi |
|---|---|
| Volumes | Conserver les données après recréation d'un conteneur. Le test de persistance ne remplace pas une sauvegarde. |
| Healthchecks | Attendre que les bases répondent avant de démarrer l'application. Le code traite aussi leurs indisponibilités ultérieures. |
| Secrets | Fichiers générés dans .secrets, absents de l'archive de code. .env.example ne contient que des paramètres non sensibles. |
| Profils | ops pour les opérations ponctuelles, admin pour les outils d'administration, search pour Elasticsearch et Kibana. |
| Réseaux | Bases sur réseau interne ; interfaces sur réseau frontal avec publication en 127.0.0.1. |

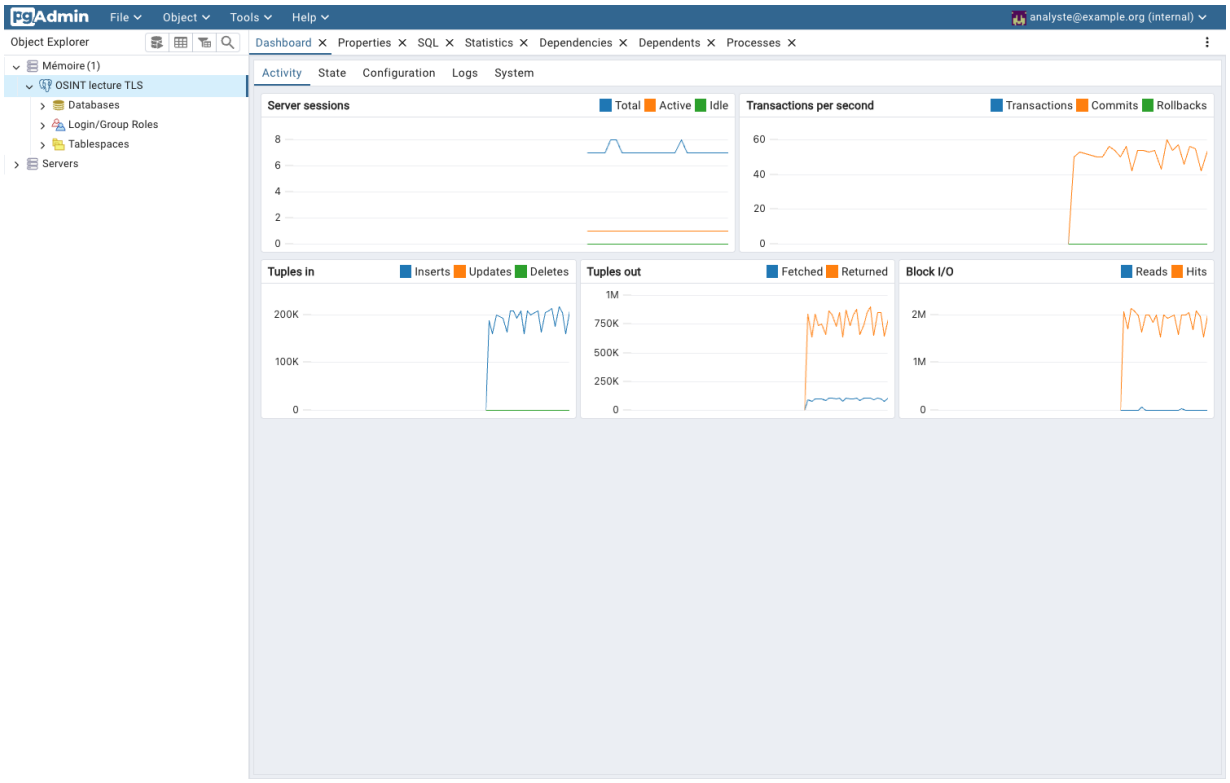
# 5. Sécurité et confidentialité

| Dimension | Réalisé et vérifié | Limite |
|---|---|---|
| Transport | TLS obligatoire vers PostgreSQL et MongoDB. HTTPS vers Elasticsearch et application. | Autorité locale et certificats de laboratoire ; exception navigateur limitée aux captures. |
| Accès | Comptes lecteur et écrivain séparés ; lecteur interdit d'écriture dans les deux moteurs. | Administration locale puissante ; revue des droits à organiser. |
| Textes au repos | AES-256-GCM, nonce aléatoire et identifiant authentifié. Sauvegarde AES-GCM. | Métadonnées, corpus source et disque non entièrement chiffrés ; FileVault désactivé. |
| Secrets | Aucun secret dans le Compose ni dans l'archive de remise ; clés montées selon besoin. | Pas de coffre central ni rotation automatique ; conserver une copie sûre des clés hors machine. |
| Programme développé dans le Bloc 3 | Permissions restrictives, Host et CSRF ; volume AES-256 activé et migration physique vérifiée (Volume_chiffre.json). | Pas de comptes individuels ; après montage, le processus lit les textes. Copies hors volume à inventorier. |
| Administration | Ports locaux et Mongo Express en lecture seule. | pgAdmin et Mongo Express utilisent HTTP local. Portainer reçoit le socket Docker et doit être fermé après usage. |

Les tests vérifient aussi le refus des connexions SQL et MongoDB sans TLS, l'intégrité AES-GCM et les contraintes de schéma. Ils ne constituent ni un audit de sécurité exhaustif ni une certification ISO 27001.

Lien au Bloc 1 : minimisation des métadonnées, responsabilités organisationnelles du scénario, sans comptes individuels dans le programme développé dans le Bloc 3, traçabilité des exécutions, durées proposées, registre des suppressions et examen préalable des prestataires. Le mini-site local ne constitue pas un hébergement public. Le contrôle CSRF ne remplace pas une authentification personnelle.

# 6. Capture PostgreSQL et application

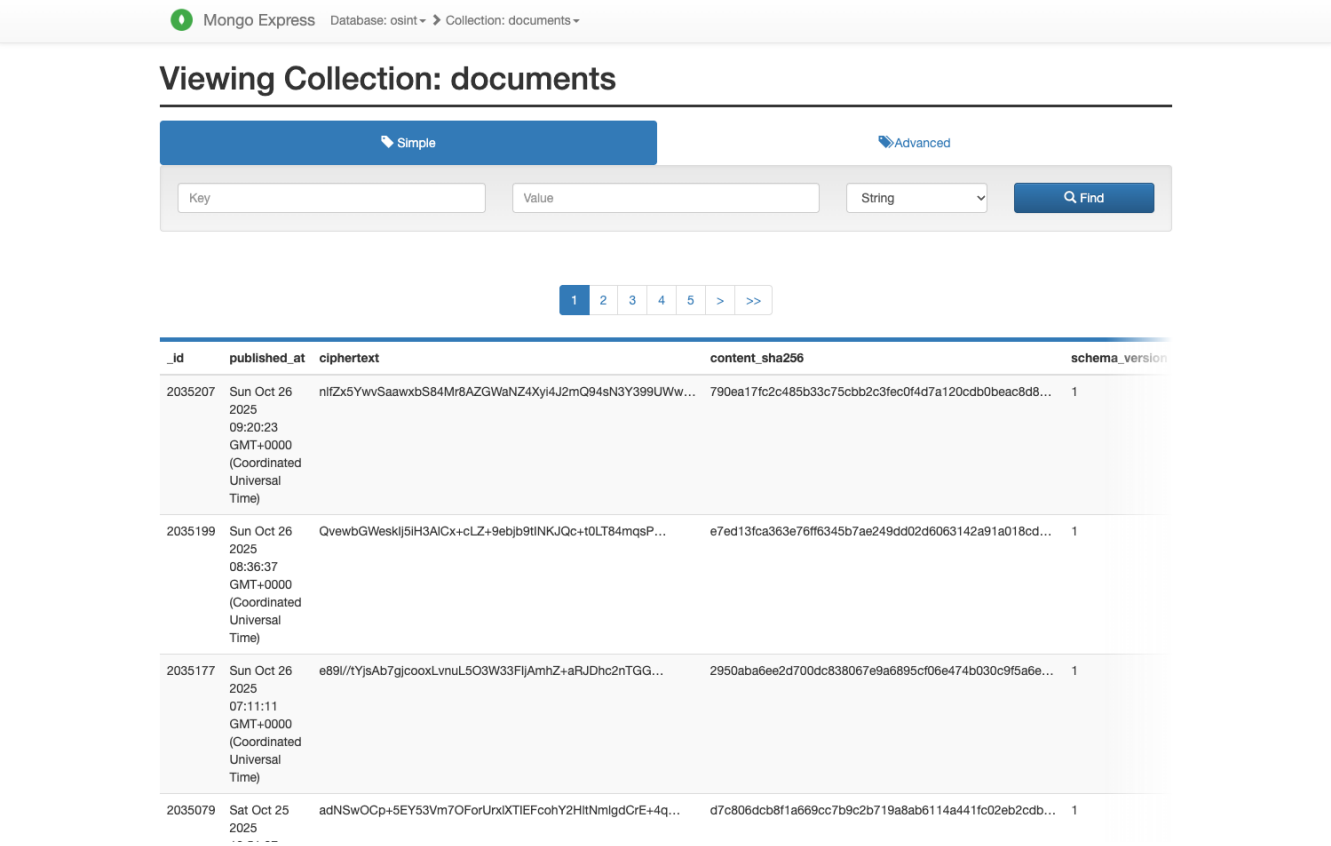



pgAdmin connecté à PostgreSQL avec le rôle de lecture. Les courbes décrivent l'activité visible pendant la capture, pas un engagement de performance.




Application Bloc 2 authentifiée, capturée pendant les essais de stockage. Les compteurs visibles concernent ce test. La capture de raccordement actualisée figure en page 18.

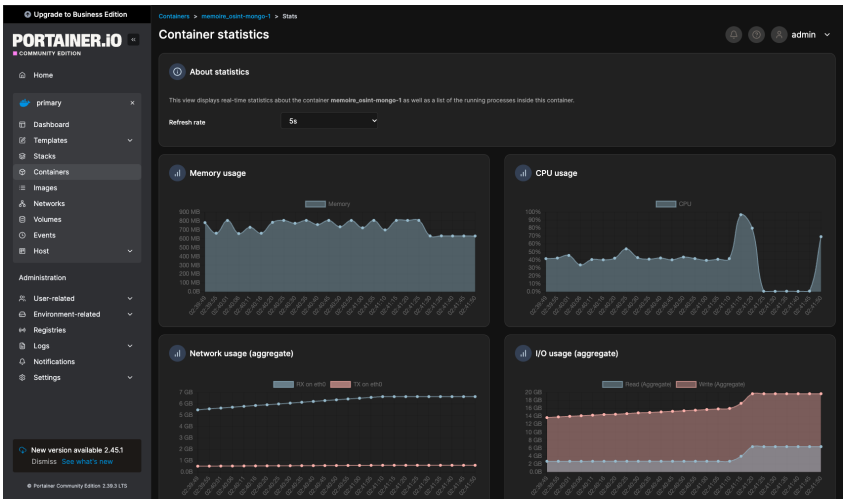
# 6. Capture Mongo Express




La collection documents expose l'identifiant commun, la date UTC, le texte chiffré et l'empreinte. Le texte intégral n'apparaît pas en clair dans l'interface de base. Les colonnes supplémentaires sont consultables dans l'original de la capture.

Mongo Express utilise osint_reader. Le contrôle d'autorisation porte sur le moteur lui-même : une tentative de suppression avec ce compte a été refusée. Masquer un bouton dans une interface ne suffit pas à contrôler les droits.

# 6. Supervision de l'infrastructure




Statistiques du conteneur MongoDB OSINT autour de l'essai. Le filtre et l'identifiant du conteneur évitent d'attribuer les ressources d'un autre projet à ce mémoire.

## Maxima échantillonnés par moteur

| Volume | Moteur | CPU max. | RAM max. |
|---|---|---|---|
| 60 000 | postgres | 10.0 % | 186.0 Mio |
| 60 000 | mongo | 55.2 % | 606.6 Mio |
| 600 000 | postgres | 47.0 % | 288.3 Mio |
| 600 000 | mongo | 93.3 % | 612.8 Mio |
| 6 000 000 | postgres | 56.5 % | 380.5 Mio |
| 6 000 000 | mongo | 104.1 % | 660.1 Mio |

Les fichiers stats_*.jsonl conservent les relevés horodatés CPU, RAM, réseau et disque des bases ; docker_stats_capture.txt complète ces traces. Le processus d'ingestion n'a pas été suivi sur toute sa durée.

Réseau et disque : compteurs cumulatifs, pas le coût d'un seul essai. CPU Docker peut dépasser 100 % avec plusieurs cœurs. Les maxima échantillonnés peuvent manquer un pic court.

Complément du 18 octobre : scripts/infrastructure.py monitor relève état, healthcheck, CPU, mémoire, réseau et disque. Alertes locales sur absence, arrêt, santé et mémoire ≥ 85 % (seuil choisi). Trois prélèvements sains et un seuil de test à 1 % sont conservés dans Preuves/Complement_20261018. Surveillance au premier plan, sans notification distante ; santé HTTP distincte de qualité des données.

# 7. Essais de volumétrie

Protocole : lots de 2 000 documents, mêmes contraintes, index, TLS et chiffrement. Les enregistrements synthétiques antérieurs sont retirés avant la taille suivante ; le corpus réel reste présent. Une exécution par taille, sans distribution statistique.

| Lignes par base | Ingestion (s) | Débit (lignes/s) | SQL = Mongo |
|---|---|---|---|
| 60 000 | 3.528 | 17007 | Vérifié |
| 600 000 | 34.318 | 17484 | Vérifié |
| 6 000 000 | 324.236 | 18505 | Vérifié |

Le temps d'ingestion mesure le traitement dans Python, de l'ouverture des clients à la validation finale. Le temps total de commande, avec démarrage du conteneur, figure séparément dans benchmarks.json. Les deux mesures ne doivent pas être mélangées.

## Nature des données

Chaque ligne représente un petit document artificiel à identifiant unique, avec texte répétitif avant chiffrement. Ces tailles ne représentent pas 6 millions d'articles réellement collectés. La charge mesure les écritures PostgreSQL et MongoDB, pas le NER ni Elasticsearch à ces volumes.

## Interprétation

Les essais de capacité ont été exécutés et les comptes ont été comparés dans les deux moteurs. La cadence proposée de 1 000 articles par jour est modeste par rapport à ces débits, mais un changement de longueur des textes, de source, de matériel ou de concurrence exige une nouvelle mesure. Les performances du poste ne constituent pas une garantie de service.

Les données synthétiques sont supprimées après contrôle. Les empreintes du code mesuré restent identifiées dans Environnement_et_version.json. La mesure ne couvre pas le téléchargement TASS, l'import via interface, la gestion des révisions ou le NER.

Versions : le manifeste distingue les empreintes des essais et celles du code livré. Après les mesures, le délai de nettoyage Mongo a été allongé, les droits du service lecteur resserrés et la présentation d'un article améliorée ; les contrôles fonctionnels ont été rejoués.

# 8. Sauvegarde et continuité

## Sauvegarde cohérente

scripts/backup_restore.py prend un verrou commun aux ingestions, capture PostgreSQL, MongoDB et les registres applicatifs, puis chiffre l'ensemble. Les tables et collections de révisions sont incluses dans les dumps. Les mutations administratives directes doivent être suspendues. L'index dérivé se reconstruit ensuite ; il n'est pas considéré comme l'autorité des données.

| Contrôle effectué | Résultat |
|---|---|
| Sauvegarde chiffrée | 47 689 885 octets ; 2.329 s |
| Restauration de contrôle | 21681 lignes SQL et 21681 documents Mongo ; 5.653 s |
| Suppression postérieure à la sauvegarde | 4 identifiant(s) réappliqué(s) avant consultation. |
| Bases actives | Conservées ; restauration dans osint_restore_test. |

## Panne et mode dégradé

La démonstration arrête PostgreSQL et MongoDB, puis consulte le même article depuis Elasticsearch. La réponse annonce le mode dégradé et le texte est comparé à celui obtenu avant la panne. Les bases sont ensuite redémarrées. Un autre contrôle arrête Elasticsearch et vérifie le chemin normal SQL/MongoDB.

La vidéo assemble deux captures réelles, avec une coupe vers le retour normal. Elle est muette et ne sert pas à mesurer un temps de basculement.

Les objectifs du Bloc 1 restent RTO 8 heures ouvrées et RPO 24 heures. Le temps mesuré ci-dessus concerne une restauration locale de contrôle, pas une reprise après perte du Mac. SQLite du programme développé dans le Bloc 3, ses instantanés, ses exports et ses checkpoints ne sont pas couverts par ce dump Bloc 2. Leur sauvegarde cohérente, une copie hors machine et un exercice de sinistre complet restent à organiser (page 17).

# 9. Droits, cohérence et reprise

| Scénario | Preuve et portée |
|---|---|
| Accès à un article | API HTTPS authentifiée, restitution du texte et de l'URL. La recherche et la qualification d'une demande réelle restent humaines. |
| Suppression | Fixture sans personne réelle : retrait de SQL, MongoDB et Elasticsearch, masquage par registre, refus de réapparition à la réingestion. |
| Restauration après suppression | Sauvegarde prise avant l'effacement de la fixture ; restauration après effacement ; absence du document vérifiée. |
| Reprise | Interruption contrôlée à 4 000 enregistrements, puis reprise du même UUID à ce point et fin à 12 000. Ce test n'est pas un arrêt brutal du processus. |
| Persistance | Recréation des deux conteneurs de bases en conservant leurs volumes ; article original identique après redémarrage. |
| Variété | Champ JSON imbriqué supplémentaire conservé et restitué lors du test de fixture. |

Les versions Bloc 2 sont effacées avec l'article et son exclusion empêche le réimport. Le scénario Droits_B3_B2.json vérifie rectification, purge des versions puis effacement sur un jeu de test isolé. Copies externes, annotations et modèle exigent une procédure coordonnée ; aucun désapprentissage n'est démontré.

Les registres de suppression et de rectification doivent accompagner le plan de reprise. Droits_B3_B2.json vérifie la réapplication d'une correction puis d'un effacement après restauration dans des bases isolées. Le registre de corrections est chiffré. Une copie ancienne du registre ne suffit pas : le script fusionne les suppressions de la sauvegarde et les suppressions locales plus récentes.

# 10. Utilisation et maintenance

## Installation locale

```
python3 scripts/infrastructure.py deploy
docker compose run --rm -T ops python -m osint.cli ingest \
  --file /data/corpus_clean.json
docker compose up -d elasticsearch
docker compose run --rm -T ops python -m osint.cli init-index
docker compose run --rm -T ops python -m osint.cli index
docker compose up -d kibana pgadmin mongo-express portainer
```

deploy prépare les secrets absents, valide Compose, construit, attend la santé des bases et de l'API puis applique la migration additive. Il conserve les volumes, sans importer de corpus. Le fichier docker/compose.yaml inclut la définition racine. Attendre Elasticsearch avant init-index. Détails et dépannage dans README.md.

## Collecte et raccordement

```
# Depuis Bloc 2, base existante :
docker compose exec -T postgres psql -U postgres -d osint \
  -v ON_ERROR_STOP=1 -f /docker-entrypoint-initdb.d/02_revisions.sql
# Préparer le volume chiffré, puis ouvrir le lanceur du Bloc 3.
# Suivi : sélectionner un job terminé puis Importer dans Bloc 2.
```

Le service importe un export figé, puis reconstruit l'index des articles. Consulter docs/Collecte_stockages_securite.md et le mode d'emploi du programme développé dans le Bloc 3 pour montage, reprise et diagnostic.

## Vérifier, diagnostiquer, arrêter

```
python3 scripts/infrastructure.py monitor --search --samples 3
docker compose run --rm -T ops python -m osint.cli status
python3 scripts/benchmark.py
# Arrêter les interfaces d'administration après usage :
docker compose stop pgadmin mongo-express portainer kibana
# Conserver les volumes lors de l'arrêt complet :
docker compose --profile admin --profile search down
```

Sur erreur : relever l'UUID, le point de reprise, le type d'erreur et la disponibilité des services. Ne pas recopier une clé dans les logs et ne pas effacer les volumes comme mesure de dépannage. Reprendre avec la même source et le même UUID, puis vérifier les comptes et les empreintes.

# 11. État des preuves et travaux restants

| État | Contenu |
|---|---|
| Testé | Ingestion, contraintes, TLS, charges synthétiques, restauration Bloc 2, lecture dégradée ; import de 5 articles, révisions, UTC, volume AES-256 et restauration des corrections/effacements sur un jeu de test. |
| Implémenté et à approfondir | Politique d'accès, répartition des autorités, chiffrement applicatif, règles de conservation et scripts d'exploitation. |
| Préparé | Trame orale de cinq minutes et 28 critères. Raccordement entre le programme développé dans le Bloc 3 et Bloc 2 testé sur 5 articles ; NER Bloc 4 exécuté sur le corpus initial uniquement. |
| À réaliser avant une exploitation réelle | Chiffrement complet du stockage, gestion et rotation des clés, copie hors machine, calendrier de purge et sauvegarde, supervision permanente et architecture multi-hôte selon disponibilité attendue. |
| Limites du rendu local | Docker Desktop testé. Plan VM Ubuntu/cloud-init et script vm/ fournis ; Multipass absent, VM non déployée. Validation juridique globale et répétition orale non attestées. |

Le critère sectoriel B2-3.4 est analysé dans le cadre de la veille documentaire OSINT : les règles propres au secret médical ou au reporting financier du cas santé ne sont pas transposées. L'analyse devra être revue si l'organisation, la finalité ou le secteur changent.

Révision du 18 octobre : déploiement reproductible et surveillance locale ajoutés ; code IaC Docker, modèle et limites explicités. Le redéploiement réutilise une installation existante. Une installation vierge sur un autre poste reste à vérifier.

Les statuts du suivi décrivent les preuves disponibles. Ils ne sont pas des notes du jury. Les sept critères de prestation orale restent préparés ; un support écrit ne démontre pas la prestation elle-même.

# 12. Sources et défense à l'oral

## Références et preuves

Grille école RNCP38777 : Bloc 2, cellules A6 à A33, soit 28 critères exacts. Consignes du directeur et complément Bloc 2 du 18 octobre : plan et diagrammes, IaC Docker, déploiement, monitoring et documentation ; oral de 5 minutes. Vidéo non actualisée dans cette révision.

Critères qualité RNCP41993 : qualité, code et traçabilité. Le mélange des identifiants RNCP reste signalé dans le pilotage.

Gouvernance OSINT Bloc 1 : responsabilités, droits, durées et limites.

Preuves/benchmarks.json, integration.json, lifecycle.json, continuity.json, backup.json, restore.json et Environnement_et_version.json : résultats datés et versions.

## Documentation officielle consultée le 4 octobre 2026

Compose : dépendances

Compose : secrets

PostgreSQL : TLS

MongoDB : TLS

Elasticsearch : HTTPS

ANSSI : conteneurs Docker

ANSSI : hygiène informatique

ANSSI : sauvegarde

## Oral : les cinq minutes

Besoin 30 s ; flux et stockages 60 s ; révisions et UTC 40 s ; sécurité 40 s ; mesures 40 s ; panne et restauration 60 s ; limites 30 s. Total prévu : 300 s. Préparer trois réponses : pourquoi ces stockages, ce que prouvent les 6 millions, ce qui reste possible pendant une panne.

# 13. Cartographie de protection et reprise

Chaque copie a un rôle, une protection et un périmètre de reprise propres. Un volume persistant ou un miroir sur le même Mac ne constitue pas une sauvegarde hors machine.

| Copie | Protection et reprise | Portée |
|---|---|---|
| SQLite du programme développé dans le Bloc 3 et WAL | Permissions 0700/0600 ; volume chiffré requis par le lanceur. Checkpoints durables. | Clair pour le processus après montage. Volume AES-256 et migration vérifiés ; copies externes distinctes. |
| Miroir et exports du programme développé dans le Bloc 3 | Instantané SQLite vérifié et exports atomiques ; miroir de lecture seule. | Mêmes hôte et volume. Sauvegarde coordonnée externe non testée. |
| Bases et révisions Bloc 2 | TLS, rôles ; textes Mongo AES-GCM. Dumps Bloc 2 chiffrés, restauration isolée. | Métadonnées non entièrement chiffrées au repos ; clés séparées. |
| Registres de Bloc 2 et du programme développé dans le Bloc 3 | Exclusions durables ; rectifications chiffrées Bloc 2, synchronisation avant exposition. | Réappliquer les décisions après restauration ; vérifier les copies externes. |
| Elasticsearch et NER | Index articles dérivé, reconstruction depuis les autorités ; NER Bloc 4 distinct. | Les résultats NER ne sont réutilisables que pour le bon texte et modèle. |
| Copies hors périmètre | Inventaire et suppression coordonnée : téléchargements, sources, annotations, modèles. | Aucune purge distante ni désapprentissage démontrés. |

Pour une sauvegarde du programme développé dans le Bloc 3 : suspendre les écritures, prendre un instantané SQLite cohérent (API backup, pas une simple copie du WAL actif), conserver exports, configuration et registres, puis tester la reprise. Cette procédure est proposée ; aucun essai de sauvegarde du programme développé dans le Bloc 3 hors machine n'est revendiqué.

Le script Bloc 2 restaure les deux bases dans osint_restore_test et réapplique les suppressions. Droits_B3_B2.json atteste la réapplication des corrections et suppressions après restauration sur un jeu de test isolé.

# 14. Raccordement depuis l'Observatoire




Capture réelle : rejeu de l'import de 5 articles depuis un job validé, sans doublon. Demonstration_raccordement_Observatoire.mp4 complète la vidéo de panne. Cette capture précède le chiffrement du volume. Son activation et la migration sont attestées séparément par Volume_chiffre.json.

| Vérification | Preuve et limite |
|---|---|
| Concordance de données | Contrat_collecte_TASS.json : 5 articles rapprochés, 21 681 articles SQL/Mongo/ES, 39 511 mentions NER conservées. |
| Révisions et dates | Revisions_collecte_TASS.json : ancienne version déchiffrable, rejeu unique, journée UTC inclusive et effacement des archives. |
| Accès et intégrité | Integration_collecte_TASS.json : 8 contrôles réussis, TLS, rôles, contraintes et chiffrement authentifié. |
| Restauration Bloc 2 | Collecte_TASS/restore.json : 21 681 articles dans chaque base isolée et 4 exclusions réappliquées. |
| Vidéo et version | Raccordement_Observatoire.json : journal, empreintes et état final de la capture ; capture locale, même hôte. |

La collecte, l'import et le NER sont trois opérations distinctes. Le petit lot démontre le raccordement technique ; il ne prouve ni l'exhaustivité des archives, ni une qualité NER sur les textes récents.

# 15. Correspondance ANSSI : conteneurs

Le critère B2-3.2 est rapproché de la fiche ANSSI-FT-082, version 1.0 du 23 septembre 2020. Les références ci-dessous sont des recommandations sélectionnées et adaptées à Docker Desktop. La matrice complète, avec preuves et écarts, est dans docs/Correspondance_ANSSI_Bloc2.md. Aucune conformité exhaustive ni certification ISO n'est revendiquée.

| Référence et objectif | Configuration et preuve | État et limite |
|---|---|---|
| R1, R2 : accès à l'hôte | Compose sans privileged ni devices ; montages explicites. | Revue de configuration. Portainer conserve le socket Docker et une forte autorité. |
| R3, R4, R5 : réseau | data_net interne, front_net ; pas de network_mode host ni ports publiés pour les bases. | Cloisonnement partiel : réseaux partagés, pas un réseau par connexion ; daemon non audité. |
| R6, R9 : isolation | Pas de pid, ipc ou cgroup_parent partageant l'hôte dans Compose. | Configuration relue ; namespaces et cgroups actifs non audités dans cette revue. |
| R7, R8 : privilèges | Aucun userns-remap ni cap_drop explicite. Ops utilise UID 0. | Durcissement à approfondir ; séparation des rôles des bases distincte des privilèges système. |
| R10, R11 : ressources | mem_limit par service ; mesures RAM dans stats_*.jsonl. | Mémoire limitée et mesurée. Aucun quota CPU explicite dans Compose. |
| R12, R13 : écritures | app en read_only ; tmpfs /tmp et état monté ro. | Protection ciblée du service de lecture ; autres services et zones temporaires à revoir. |
| R14, R15 : stockage | Volumes dédiés aux bases ; secrets selon service. application_permissions.json. | Accès lecteur réduit testé ; quota disque absent, opérateur et administration puissants. |
| R16 : journaux | Logs d'ingestion et docker stats ; consultation locale. | Collecte locale attestée. Aucun export automatique des logs hors de l'hôte. |

Les tests SQL/Mongo de droits, TLS et intégrité sont conservés avec leur version. La lecture du Compose ne prouve pas à elle seule la sécurité du moteur Docker ou du Mac.

# 16. Correspondance ANSSI : exploitation

Références : Guide d'hygiène informatique (janvier 2017) et Les essentiels, Sauvegarde des systèmes d'information (v1.1, décembre 2023). La correspondance distingue les mécanismes testés, les configurations relues et les actions d'exploitation restantes.

| Référence et objectif | Mesure et preuve | Limite et action |
|---|---|---|
| Hygiène 8, 9 : identités et accès | Rôles lecteur/écrivain ; refus d'écriture dans integration.json et permissions ES ciblées. | Comptes de service partagés ; attribution personnelle et revue périodique à organiser. |
| Hygiène 11, 12 : secrets | Génération locale dans scripts/setup.py ; secrets hors dépôt et montés par service. | Fichiers privés ne constituent pas un coffre chiffré. Rotation et récupération sûre à organiser. |
| Hygiène 19, 29 : réseau et administration | Réseau de données interne ; interfaces sur 127.0.0.1 ; profil admin ponctuel. | Pas de poste d'administration dédié ; HTTP local pgAdmin/Mongo Express, socket Portainer. |
| Hygiène 30 : poste nomade | Textes Mongo et sauvegardes AES-GCM ; catalogue du programme du Bloc 3 sur volume AES-256. | Volume_chiffre.json vérifie migration et intégrité. Métadonnées Bloc 2 et copies externes non intégralement chiffrées. |
| Hygiène 34, 35 : mises à jour | Versions et empreintes dans Compose et Environnement_et_version.json. | Inventaire présent ; procédure de correctifs, veille et fins de support à formaliser. |
| Hygiène 36 : journalisation | Logs des opérations, erreurs, checkpoints et versions des essais. | Rétention, horodatage commun et journalisation de tous les accès à vérifier ; pas de centralisation distante. |
| Hygiène 37 ; Essentiels, partie 2 | Dumps Bloc 2 chiffrés ; restore.json et Droits_B3_B2.json : restauration isolée, corrections et exclusions réappliquées. | Essais locaux sur un jeu de test. Fréquence automatisée, copie hors ligne et sinistre complet à organiser. |
| Hygiène 38 : contrôles | Tests de droits/TLS, intégrité, reprise et lecture dégradée. | Contrôles ciblés documentés ; aucun audit externe ni garantie d'absence de vulnérabilité. |

Le rapprochement documentaire est réalisé. Les écarts techniques restent inscrits dans le suivi : durcissement des privilèges, protection des copies, gestion des clés, sauvegarde hors machine et journalisation distante.